CIS*3110 Operating Systems

Week 10

Lecture #16 Concurrency problems

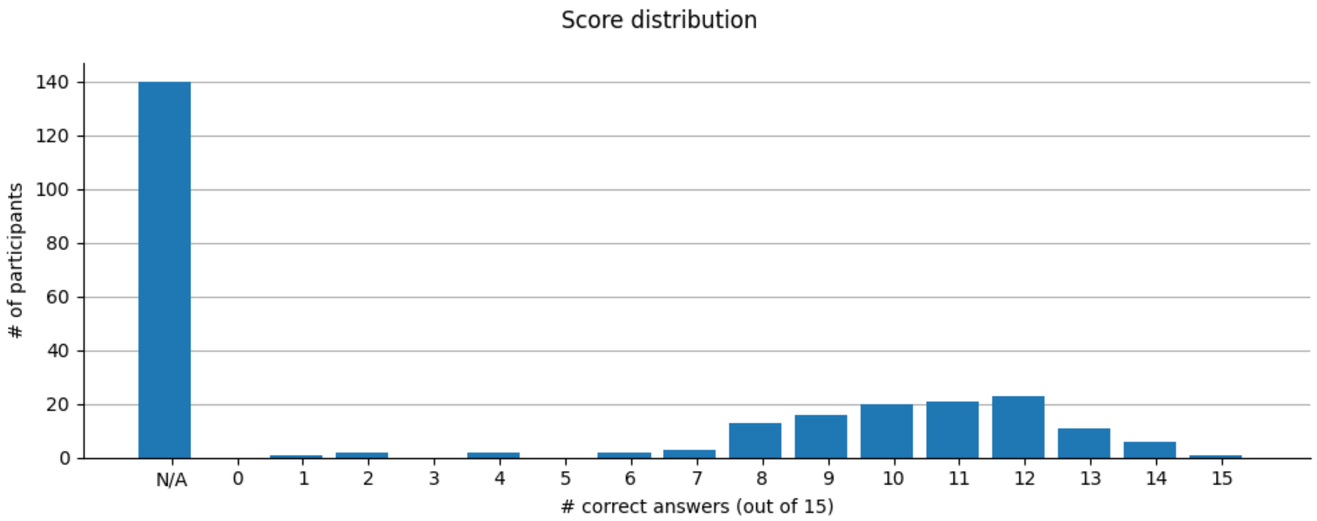
Prof. Yulia Kotseruba

March 12, 2026

Quiz 14

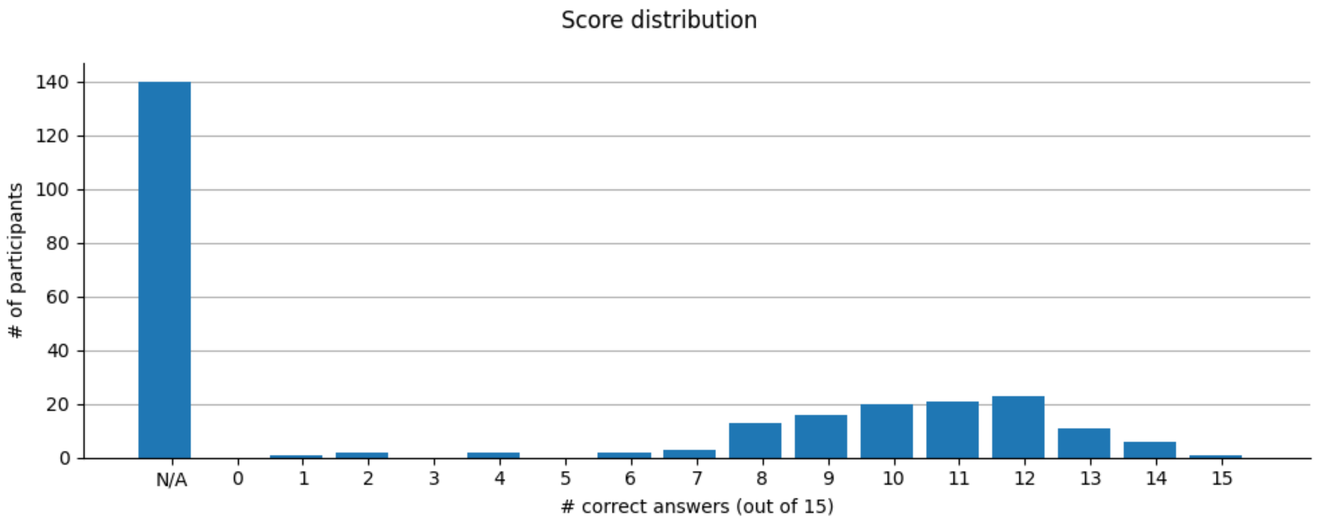
```
32-bit virtual address

2^14 (16KB) page size

2^12 (4KB) RAM
```

- **62%** average (down from 72% in quiz 13)
- 3 hardest questions:
  - (13%) Given the memory parameters above, how many bits are needed to encode a page number? a) 10 b) 18 c) 20 d) none of the above
  - (28%) What is printed when this code executes? a) 1 2 3 b) 2 1 3 c) 3 1 2 d) any
  - (42%) What page should be evicted from the cache for best performance? PB - present bit, DB - dirty bit a) PB=0, DB=1 b) PB=1, DB=0 c) PB=0, DB=0 d) PB=1,DB=1



Quiz 14

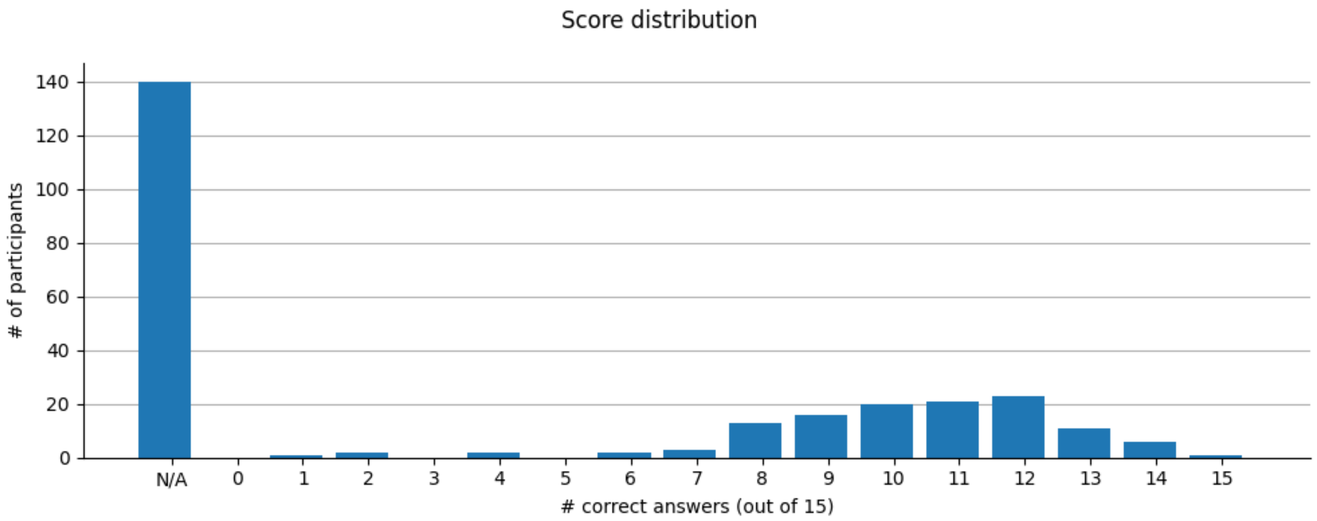
- **62%** average (down from 72% in quiz 13)
- 3 hardest questions:
  - (13%) Given the memory parameters above, how many bits are needed to encode a page number?
  - (28%) What is printed when this code executes? a) 1 2 3 b) 2 1 3 c) 3 1 2 d) any
  - (42%) What page should be evicted from the cache for best performance? PB - present bit, DB - dirty bit a) PB=0, DB=1 b) PB=1, DB=0 c) PB=0, DB=0 d) PB=1,DB=1




```c
void *inc(void *arg) {
    static int val = 0;
    val++;
    printf("%d\n", val);
    return NULL;
}

int main(void) {
    int n = 3;
    pthread_t *t = malloc(n * sizeof *t);

    for (int i = 0; i < n; ++i){
        pthread_create(&t[i], NULL, inc, NULL);
        pthread_join(t[i], NULL);
    }
    free(t);
    return 0;
}
```

Quiz 14

- **62%** average (down from 72% in quiz 13)
- 3 hardest questions:
  - (13%) Given the memory parameters above, how many bits are needed to encode a page number?
  - (28%) What is printed when this code executes? a) 1 2 3 b) 2 1 3 c) 3 1 2 d) any
  - (42%) What page should be evicted from the cache for best performance? PB - present bit, DB - dirty bit a) PB=0, DB=1 b) PB=1, DB=0 c) PB=0, DB=0 d) PB=1,DB=1



PB=0 DB=1 - not present, can't evict

PB=0 DB=0 - not present, can't evict

PB=1 DB=1 - present, modified, expensive to evict (need to write to HD)

PB=1 DB=0 - present, unmodified, OK to evict

Recap

- Software-only locks do not work on modern hardware
- Atomic hardware instructions are the standard
  - Option #1 **test-and-set and spin** - not fair, not starvation-free, spinning wastes CPU
  - Option #2 **test-and-set and yield** - not fair, not starvation-free, less overhead
  - Option #3 **futex** - fair (uses a queue), starvation-free, efficient (one variable, few operations, minimizes kernel calls)
- Condition variables - useful for communication
  - require **both** locks and global state to work correctly

Thread synchronization

- Signaling via condition variables
  - parent and child thread coordinating (Lecture #15)
- Producer/Consumer problem (aka bounded buffer)
  - producers generate data and place them in the buffer
  - consumers read data from the buffer and do something with them
  - introduced by Dijkstra in 1972
  - led to the invention of the semaphore

Producer/Consumer Problem

- Producer thread(s) write to a shared buffer
- Consumer thread(s) read from a shared buffer
- The shared buffer is bounded
  - consumer process must wait if the buffer is empty
    - i.e. they cannot consume something that hasn’t been produced yet
  - producer process must wait if the buffer is full
    - i.e. they cannot overwrite existing items and must wait for space to open

our buffer is very small, just one integer

count keeps track of how many items are in the buffer

```
int buffer, count = 0;

void *producer(void *arg){
    for (int i = 0; i < 3; i++){
        assert(count == 0);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
    }
    return NULL;
}

void *consumer(void *arg){
    for (int i = 0; i < 3; i++){
        assert(count == 1);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
    }
    return NULL;
}
```

producer generates three items

checks if there is space in the buffer

updates the buffer

increments the count to show that an item is available

consumer consumes 3 items

checks if buffer is not empty

grabs the value and decrements count to show that the item was consumed

main creates two threads (consumer and producer) and waits for them to finish

```
int main(){
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

```c
int buffer, count = 0;

void *producer(void *arg){
    for (int i = 0; i < 3; i++){
        assert(count == 0);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
    }
    return NULL;
}

void *consumer(void *arg){
    for (int i = 0; i < 3; i++){
        assert(count == 1);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
    }
    return NULL;
}
```

cp_v1.c

let's run this code

```
>>> ./cp_v1
Produced: 0
Consumed: 0
Produced: 1
Consumed: 1
Assertion failed: (count==1) function
producer
```

doesn't work. Why?

race conditions in both producer and consumer

what can we do?

locks and condition variables around CS

```c
int main(){
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

```c
int buffer, count = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

initialize a lock and cond var

acquire a lock and wait for the signal

produce and signal again

wait for producer

consume and signal

cp_v2.c

```c
int buffer, count = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

```
>>> ./cp_v2
Produced: 0
Consumed: 0
Produced: 1
Consumed: 1
Produced: 2
consumed: 2
```

this works for 2 threads

will it work for more threads?

what if we have 2 consumers and 1 producer

 cp_v2.c

C1	C2	P1

buffer = 0

count = 0

C1 starts

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 C2 P1

buffer = 0

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 starts

gets the lock

C1 C2 P1

buffer = 0

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 starts

gets the lock

and waits for the signal

C1	C2	P1

buffer = 0

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 starts

gets the lock

and waits for the signal

pthread_cond_wait releases the lock while blocking the waiting thread

C1 C2 P1

buffer = 0

count = 0

C2 starts

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 C2 P1

buffer = 0

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C2 starts gets the lock

C1	C2	P1

buffer = 0

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C2 starts

gets the lock

and waits…

C1	C2	P1

buffer = 0

count = 0

producer starts gets the lock

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1	C2	P1

buffer = 1

count = 1

producer starts

gets the lock

updates the buffer and count

signals

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1	C2	P1

buffer = 1

count = 1

producer starts

gets the lock

updates the buffer and count

signals and goes back to waiting for consumers

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1	C2	P1

buffer = 1

count = 1

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

? ?

only one thread can be awoken by the signal

which one?

C1	C2	P1

buffer = 1

count = 1

say C1 woke up

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 C2 P1

buffer = 1

count = 0

say C1 woke up

updated count

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1	C2	P1

buffer = 1

count = 0

say C1 woke up

updated count

and signaled

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 C2 P1

buffer = 1

count = 0

say C1 woke up

updated count

and signaled

will P1 or C2 wake up?

?

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

?

C1	C2	P1

buffer = 1

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

?

it would make more sense for P1 to wake up because the buffer is empty

but we have no control here…

say C1 woke up

updated count

and signaled

will P1 or C2 wake up?

?

C1 C2 P1

buffer = 1

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

it would make more sense for P1 to wake up because the buffer is empty

but we have no control here…

unfortunately for us, C2 wakes up instead

checks that count is 0

C1 C2 P1

buffer = 1

count = 0

unfortunately for us, C2 wakes up instead

checks that count is 0 and goes back to sleep

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

C1 C2 P1

buffer = 1

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

unfortunately for us, C2 wakes up instead

checks that count is 0 and goes back to sleep

~~the end~~ deadlock

why did this happen?

C1 C2 P1

buffer = 1

count = 0

```
int buffer, count = 0;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&cv, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&cv, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

unfortunately for us, C2 wakes up instead

checks that count is 0 and goes back to sleep

~~the end~~ deadlock

why did this happen?

problem: we don't have control over which threads wake up

solution: separate cond vars for producers and consumers

```c
int buffer, count = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t fill = PTHREAD_COND_INITIALIZER;
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&empty, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&fill);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&fill, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

two condition variables: fill and empty

producer waits for the buffer to empty

and signals when the buffer is full

this will work for many consumers and many producers working together

consumer waits for the buffer to fill

and signals when the buffer is empty

cp_v3.c

```c
int buffer, count = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t fill = PTHREAD_COND_INITIALIZER;
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 1) pthread_cond_wait(&empty, &m);
        buffer = i;
        count = 1;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&fill);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&fill, &m);
        int v = buffer;
        count = 0;
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

what if we want a larger buffer?

cp_v3.c

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t full  = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == MAX) pthread_cond_wait(&empty, &m);
        put(i);
        printf("Produced: %d\n", i);
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&full, &m);
        int v = get();
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&m);
   }
    return NULL;
}
```

updating the code for MAX size buffer

most of the code will stay the same

just need to compare count to MAX or to 0

this will put the threads to sleep only when the buffer is completely full or empty -> good for concurrency

also need new functions for adding items to the buffer and removing them: put() and get()

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t full  = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == MAX) pthread_cond_wait(&empty, &m);
        put(i);
        printf("Produced: %d\n", i);
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&full, &m);
        int v = get();
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&m);
   }
    return NULL;
}
```

```c
#define MAX 5
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}

int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

use_ptr and fill_ptr keep track of current location in the buffer

put adds a value to the buffer and increments both fill_ptr and count

get decrements use_ptr and count and returns the deleted value from the buffer

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t full  = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == MAX) pthread_cond_wait(&empty, &m);
        put(i);
        printf("Produced: %d\n", i);
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&full, &m);
        int v = get();
        printf("Consumed: %d\n", v);
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&m);
   }
    return NULL;
}
```

```c
#define MAX 5
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

are we done now?

this works but there are better options

```c
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

Summary so far

- **Consumer/producer problem** (aka bounded buffer problem)
- A very common problem in concurrent software development
- Our working solution
  - 2 condition variables - 1 for consumers and 1 for producers
  - 1 mutex
  - a bounded size buffer
  - any number of consumer/producer threads
  - no deadlocks

Semaphore

- Introduced by Dijsktra in 1968
- A single primitive for synchronization combining
  - a mutex lock
  - a condition variable
  - and more

Semaphore

- an integer
- sem_init - initializes a semaphore to some initial value
- sem_wait()
  - decrements the value of s by 1
  - waits if semaphore is negative
- sem_post()
  - increments the value of s by 1
  - wakes one of the threads waiting (if any)
    - waiting processes are in a queue, but waking order is not guaranteed

Lock with semaphore

```
int counter = 0;
sem_t sem;

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {

        counter++;

    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("counter = %d\n", counter);
    return 0;
}
```

each worker increments a common counter value for 1000 iterations

because counter++ is not atomic there is a race condition here

this is code from the previous lecture

main creates two worker threads

Lock with semaphore

```
int counter = 0;
sem_t sem;

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {

        counter++;

    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("counter = %d\n", counter);
    return 0;
}
```

each worker increments a common counter value for 1000 iterations

because counter++ is not atomic there is a race condition here

before we used locks to protect CS

today we will do it with a semaphore

this is code from the previous lecture

main creates two worker threads

Lock with semaphore

```
int counter = 0;
sem_t sem;

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    sem_init(&sem, 0, ?);
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    sem_destroy(&sem);
    printf("counter = %d\n", counter);
    return 0;
}
```

declare a semaphore

put sem_wait and sem_post around CS

same as locks before

before we used locks to protect CS

today we will do it with a semaphore

need to initialize the semaphore

also destroy the semaphore at the end

Lock with semaphore

```
int counter = 0;
sem_t sem;

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    sem_init(&sem, 0, ?);
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    sem_destroy(&sem);
    printf("counter = %d\n", counter);
    return 0;
}
```

sem_wait decrements and waits if sem < 0

sem_post increments and wakes a thread (if any are waiting)

what initial value of the semaphore would make this work?

Lock with semaphore

```
int counter = 0;
sem_t sem;

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    sem_init(&sem, 0, 1);
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    sem_destroy(&sem);
    printf("counter = %d\n", counter);
    return 0;
}
```

sem_wait decrements and waits if sem < 0

sem_post increments and wakes a thread (if any are waiting)

what initial value of the semaphore would make this work?

thread A

sem = 1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A gets here first

case #1 one thread

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A gets here first

decrements sem

case #1 one thread

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A gets here first

decrements sem

and proceeds to CS

case #1 one thread

thread A

sem = 1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A gets here first

decrements sem

and proceeds to CS

then increments sem

case #1 one thread

thread A

sem = 1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A gets here first

decrements sem

and proceeds to CS

then increments sem

and releases the lock

no other threads are waiting,
nothing else to do here

case #1 one thread

thread A

sem = 1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

B is interrupted

A gets here first

case #2 two threads

thread A

sem = 1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A calls sem_wait

B is interrupted

case #2 two threads

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A calls sem_wait
decrements sem

B is interrupted

case #2 two threads

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A calls sem_wait
decrements sem
goes to CS

B is interrupted

case #2 two threads

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A calls sem_wait
decrements sem
goes to CS
and is interrupted

B is interrupted

case #2 two threads

thread A

sem = -1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A calls sem_wait
decrements sem
goes to CS
and is interrupted

B wakes up
decrements sem

case #2 two threads

thread A

sem = -1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A calls sem_wait
decrements sem
goes to CS
and is interrupted

B wakes up
decrements sem
sem < 0 -> sleep

case #2 two threads

thread A

sem = -1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A wakes up

increments counter

calls sem_post

case #2 two threads

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A wakes up

increments counter

calls sem_post

increments sem

case #2 two threads

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

A wakes up

increments counter

calls sem_post

increments sem

wakes B

exits

case #2 two threads

thread A

sem = 0

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

B wakes up

sem>=0 -> can go to CS

increments counter

case #2 two threads

thread A

sem = 1

thread B

```
void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}
```

B wakes up
sem>=0 -> can go to CS

increments counter

calls sem_post

restores sem

case #2 two threads

Condition variable with semaphore

- Semaphores can also be used as condition variables

```c
volatile int done = 0;

void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    while (!done) ;
    printf("parent: end\n");
    return 0;
}
```

threads_talking.c

thread prints and updates a variable

this is another example from the previous lecture

how can we use a semaphore here?

main prints

main creates a thread

main spins waiting for the variable to be updated

main prints and exits

```c
volatile int done = 0;

void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    while (!done) ;
    printf("parent: end\n");
    return 0;
}
```

threads_talking.c

how can we use a semaphore here?

```c
sem_t s;

void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, ?);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

threads_talking.c

what should s be initialized as?

how can we use a semaphore here?

sem_wait decrements and waits if sem < 0

sem_post increments and wakes a thread (if any are waiting)

```c
sem_t s;

void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

threads_talking.c

what should s be initialized as?

how can we use a semaphore here?

sem_wait decrements and waits if sem < 0

sem_post increments and wakes a thread (if any are waiting)

sem = 0

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

parent calls sem_wait

case #1 parent first

sem = -1

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

parent calls sem_wait

decrements sem

case #1 parent first

sem = -1

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

parent calls sem_wait

decrements sem

sem < 0 -> sleeps

case #1 parent first

sem = 0

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

child will increment sem

case #1 parent first

sem = 0

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

child will increment sem

and wake parent

case #1 parent first

sem = 0

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

sem >= 0 -> parent will unblock, print and exit

case #1 parent first

sem = 0

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

child calls sem_post first

case #2 child first

sem = 1

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

child calls sem_post first

increments sem

no threads to wake

and exits

case #2 child first

sem = 1

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

parent calls sem_wait

case #2 child first

sem = 0

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

parent calls sem_wait

decrements sem

case #2 child first

sem = 0

child

parent

```
void *child(void *arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

parent calls sem_wait

decrements sem

sem >= 0 -> can proceed

case #2 child first

Producer/Consumer with Semaphore

- Producer thread(s) write to a shared buffer
- Consumer thread(s) read from a shared buffer
- Can use two semaphores
  - one for consumers and one for producers

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t full  = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == MAX) pthread_cond_wait(&empty, &m);
        put(i);
        printf("Produced: %d\n", i);
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&full, &m);
        int v = get();
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&m);
        printf("Consumed: %d\n", v);

   }
    return NULL;
}
```

```c
#define MAX 5
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

this is our working solution with mutexes and cond variables

```c
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
   }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        printf("Consumed: %d\n", v);

  }
    return NULL;
}
```

what should we initialize empty and full to?

```c
#define MAX 5
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

this is our working solution with mutexes and cond variables

lets replace them with semaphores

```c
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
   }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        printf("Consumed: %d\n", v);

  }
    return NULL;
}
```

what should we initialize empty and full to?

empty to MAX (it will be decremented), full to 0 (it will be incremented)

```c
#define MAX 5
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

this is our working solution with mutexes and cond variables

lets replace them with semaphores

```c
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

P1 P2

full = 0
empty = 10

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
    }
    return NULL;
}
```

two producers call sem_wait at the same time

```
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

P1 P2

full = 0
empty = 8

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
    }
    return NULL;
}
```

two producers call sem_wait at the same time

decrement empty

```
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

P1 P2

full = 0
empty = 8

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
    }
    return NULL;
}
```

both can proceed to put(i)

```
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

P1 P2

full = 0
empty = 10

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
    }
    return NULL;
}
```

two producers call put at the same time

P1 puts value into buffer first

```
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

P1 P2

full = 0
empty = 10

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
    }
    return NULL;
}
```

two producers call put at the same time

P1 puts value into buffer first

and is interrupted

```
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

P1

P2

full = 0
empty = 10

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
   }
    return NULL;
}
```

P2 starts and overwrites the buffer with its own value

```
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

P1 P2

full = 0
empty = 10

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full):
   }
    return NULL;
}
```

P2 starts and overwrites the buffer with its own value

we forgot mutual exclusion!

```
void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int v = get();
        sem_post(&mutex);
        sem_post(&empty);
        printf("Consumed: %d\n", v);

  }
    return NULL;
}
```

what should mutex be initialized to?

```c
#define MAX 5
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
```

need to add mutexes around CS

```c
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&mutex);
        sem_post(&full);
   }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int v = get();
        sem_post(&mutex);
        sem_post(&empty);
        printf("Consumed: %d\n", v);

  }
    return NULL;
}
```

```c
#define MAX 5
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}

int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

need to add mutexes around CS

what should mutex be initialized to?

this was a few slides back, init to 1

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&mutex);
        sem_post(&full);
   }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int v = get();
        sem_post(&mutex);
        sem_post(&empty);
        printf("Consumed: %d\n", v);

  }
    return NULL;
}
```

does mutex placement matter?

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full);
        sem_post(&mutex);
  }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("Consumed: %d\n", v);

  }
    return NULL;
}
```

does mutex placement matter?

what if we put mutexes around full/empty instead?

P

C

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full);
        sem_post(&mutex);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("Consumed: %d\n", v);

    }
    return NULL;
}
```

consumer runs first

gets the lock

P

C

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full);
        sem_post(&mutex);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("Consumed: %d\n", v);

    }
    return NULL;
}
```

consumer runs first

gets the lock

calls sem_wait on full and waits

P C

producer runs next

consumer runs first

gets the lock

calls sem_wait on full and waits

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full);
        sem_post(&mutex);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("Consumed: %d\n", v);

    }
    return NULL;
}
```

P

C

producer runs next and gets stuck waiting for the mutex

consumer runs first

gets the lock

calls sem_wait on full and waits

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full);
        sem_post(&mutex);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("Consumed: %d\n", v);

    }
    return NULL;
}
```

P

C

producer runs next and gets stuck waiting for the mutex

deadlock again!

consumer runs first

gets the lock

calls sem_wait on full and waits

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        printf("Produced: %d\n", i);
        sem_post(&full);
        sem_post(&mutex);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 3; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int v = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("Consumed: %d\n", v);

    }
    return NULL;
}
```

Summary

- Producer/consumer problem
  - multiple producer threads put values into the shared buffer
  - multiple consumer threads get values from the shared buffer
- Semaphores
  - binary - only 1 and 0 values allowed -> often used for mutexes
  - counting - any integer values allowed -> mutexes, condition vars
  - In current Linux kernel, POSIX semaphores are >= 0
    - this doesn’t change the logic

Reading

- Chapter 30 Condition variables
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf
- Chapter 31 Semaphores
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-cv
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-sema

Timeline update

- Next lecture will be the last on Concurrency
- March 19th will begin Persistence

Quiz 15

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999